Introduksjon til Databaser

Grunnleggende konsepter for deg som er ny til databaser

La oss begynne! 🗄️

Hva skal vi lære i dag?

- 📋 **Tabeller** – Grunnsteinen i en database
- 🔤 **Kolonnetyper** – Hvilke typer data kan vi lagre?
- 🔗 **Relasjoner** – Hvordan henger data sammen?
- 🔑 **Primær- og fremmednøkler** – Kobling mellom tabeller
- 🔍 **INNER JOIN** – Hente data fra flere tabeller samtidig

Tabeller

Hva er en tabell?

En tabell er den grunnleggende strukturen for å lagre data i en database.

En tabell består av:

- **Rader** (også kalt poster/records) – én enhet av data
- **Kolonner** (også kalt felt/fields) – en egenskap ved dataene
- **Celler** – verdien der en rad og kolonne møtes

Tenk på en tabell som et **Excel-ark** – men med strenge regler for hva slags data som kan legges inn!

Eksempel: Tabell med studenter

```sql
-- Opprette tabellen
CREATE TABLE studenter (
    id          INTEGER,
    fornavn     VARCHAR(50),
    etternavn   VARCHAR(50),
    epost       VARCHAR(100),
    fodselsar   INTEGER
);
```

id	fornavn	etternavn	epost	fodselsar
1	Kari	Nordmann	kari@skole.no	2005
2	Ola	Hansen	ola@skole.no	2004
3	Ingrid	Berg	ingrid@skole.no	2005

Kolonnetyper

Hva er kolonnetyper?

Hver kolonne i en tabell må ha en **datatype** som bestemmer hva slags verdier som kan lagres der.

Dette er viktig fordi det:

- Sikrer at riktig type data lagres
- Gjør databasen mer effektiv
- Forhindrer feil (f.eks. tekst der det skal være tall)

Vanlige kolonnetyper

Tekst

Type	Beskrivelse
`CHAR(n)`	Fast lengde tekst
`VARCHAR(n)`	Variabel lengde tekst (maks n tegn)
`TEXT`	Ubegrenset tekst

Tall

Type	Beskrivelse
`INTEGER`	Heltall (…-1, 0, 1, 2…)

Dato og tid

Type	Beskrivelse
`DATE`	Dato (YYYY-MM-DD)
`TIME`	Klokkeslett
`DATETIME`	Dato og tid kombinert

Andre

Type	Beskrivelse
`BOOLEAN`	Sann/Usann

Kolonnetyper i praksis

```sql
CREATE TABLE produkter (
    id            INTEGER,          -- heltall: 1, 2, 3 ...
    navn          VARCHAR(100),     -- tekst: maks 100 tegn
    beskrivelse   TEXT,             -- lang tekst uten grense
    pris          DECIMAL(10, 2),   -- f.eks. 199.90
    på_lager      BOOLEAN,          -- TRUE eller FALSE
    opprettet     DATE              -- f.eks. 2024-01-15
);
```

💡 **Tips:** Velg alltid den typen som passer best til dataene dine. Bruk ikke `TEXT` når `VARCHAR(50)` holder, og ikke `FLOAT` til pengebeløp (bruk `DECIMAL` for å unngå avrundingsfeil!).

Relasjoner

Hva er relasjoner?

En relasjonsdatabase lar oss koble tabeller sammen slik at vi unngår å gjenta de samme dataene mange ganger.

Uten relasjoner (dårlig):

ordre_id	kunde_navn	kunde_epost	vare
1	Kari N.	kari@skole.no	Bok
2	Kari N.	kari@skole.no	Penn
3	Ola H.	ola@skole.no	Linjal

Med relasjoner (bra):

Tabell: `kunder`

id	navn	epost
1	Kari N.	kari@skole.no
2	Ola H.	ola@skole.no

Tabell: `ordrer`

id	kunde_id	vare

Primær- og fremmednøkler

Primærnøkkel (Primary Key)

En **primærnøkkel** er en kolonne (eller kombinasjon av kolonner) som **unikt identifiserer** hver rad i en tabell.

```
CREATE TABLE studenter (
    id        INTEGER     PRIMARY KEY,  -- primærnøkkel!
    fornavn   VARCHAR(50) NOT NULL,
    etternavn VARCHAR(50) NOT NULL,
    epost     VARCHAR(100)
);
```

Regler for primærnøkler:

- Må være **unik** – ingen to rader kan ha samme verdi
- Kan **ikke** være `NULL` (tom/ukjent)
- Hver tabell bør ha **én** primærnøkkel
- Ofte brukes et automatisk incrementerende tall: `SERIAL` / `AUTO_INCREMENT`

Fremmednøkkel (Foreign Key)

En **fremmednøkkel** er en kolonne som refererer til primærnøkkelen i en annen tabell – og skaper dermed en kobling mellom tabellene.

```
CREATE TABLE klasser (
    id    INTEGER     PRIMARY KEY,
    navn  VARCHAR(50) NOT NULL
);

CREATE TABLE studenter (
    id        INTEGER     PRIMARY KEY,
    fornavn   VARCHAR(50) NOT NULL,
    klasse_id INTEGER     REFERENCES klasser(id)  -- fremmednøkkel!
);
```

🔗 `klasse_id` i `studenter`-tabellen peker på `id` i `klasser`-tabellen. Databasen sørger for at du ikke kan legge inn en `klasse_id` som ikke finnes!

Primær- og fremmednøkler visualisert

Tabell: `klasser`

id 🔑	navn
1	1A
2	1B
3	2A

Tabell: `studenter`

id 🔑	fornavn	klasse_id 🔗
1	Kari	1
2	Ola	1
3	Ingrid	2

`studenter.klasse_id` → `klasser.id`

Kari og Ola går i klasse **1A**, Ingrid går i klasse **1B**

INNER JOIN

Hva er INNER JOIN?

`INNER JOIN` lar oss hente data fra **to eller flere tabeller** i én spørring, ved å koble dem sammen på en felles nøkkel.

```
SELECT kolonne1, kolonne2, ...
FROM tabell1
INNER JOIN tabell2 ON tabell1.nokkel = tabell2.nokkel;
```

🔍 `INNER JOIN` returnerer **kun rader der det finnes en match i begge tabeller**. Rader uten kobling blir utelatt.

INNER JOIN – Eksempel

Vi ønsker å vise studentenes navn **og** klassen de tilhører:

```
SELECT
    studenter.fornavn,
    studenter.etternavn,
    klasser.navn AS klasse
FROM studenter
INNER JOIN klasser ON studenter.klasse_id = klasser.id;
```

Resultat:

fornavn	etternavn	klasse
Kari	Nordmann	1A
Ola	Hansen	1A
Ingrid	Berg	1B

INNER JOIN – Visualisert

studenter

id	fornavn	klasse_id
1	Kari	**1**
2	Ola	**1**
3	Ingrid	**2**
4	Per	NULL ❌

⟺

klasser

id	navn
1	1A
2	1B
3	2A ❌

Resultat (INNER JOIN)

fornavn	klasse
Kari	1A
Ola	1A
Ingrid	1B

Per (ingen klasse) og 2A (ingen studenter) er ikke med i resultatet

INNER JOIN med tre tabeller

Vi kan også koble sammen flere tabeller i én spørring:

```
SELECT
    studenter.fornavn,
    klasser.navn      AS klasse,
    laerere.navn      AS laerer
FROM studenter
INNER JOIN klasser ON studenter.klasse_id = klasser.id
INNER JOIN laerere ON klasser.laerer_id   = laerere.id;
```

💡 Du kan lenke så mange `INNER JOIN` som du trenger – bare sørg for at du alltid kobler via nøklene!

Oppsummering

📋 **Tabeller** Grunnstrukturen i en database. Rader = poster, kolonner = egenskaper.

🔤 **Kolonnetyper** Hver kolonne har en type: `INTEGER`, `VARCHAR`, `DATE`, `BOOLEAN`, osv.

🔗 **Relasjoner** Unngå å gjenta data – koble tabeller i stedet for å duplisere informasjon.

🔑 **Primærnøkkel** Unik identifikator for hver rad i en tabell. Kan ikke være `NULL`.

🔗 **Fremmednøkkel** Refererer til en primærnøkkel i en annen tabell og skaper en kobling.

🔍 **INNER JOIN** Henter rader fra to eller flere tabeller der det finnes match på nøklene.

Spørsmål?

Prøv gjerne å lage din egen database med tabeller, nøkler og JOIN-spørringer!

```
-- Kom i gang med din første tabell!
CREATE TABLE mine_data (
 id    INTEGER PRIMARY KEY,
 navn  VARCHAR(100) NOT NULL
);
```